Chapter 5 Application Testing

5.1 Introduction

From consumer goods like vehicles to corporate applications like banking, software systems are a necessary element of everyday life. Most people have encountered software that does not function as intended. Inaccurate software can result in several issues, such as lost revenue, wasted time, or company reputation, as well as harm or even demise. Software testing lowers the possibility of software failure while it is in use and evaluates the software's quality.

Testing is sometimes misunderstood to involve nothing more than running tests, that is, putting the software through its paces and examining the output. Software testing, as delineated in a multifaceted process including various tasks, of which test execution, including result verification, is just one. The examination process additionally involves tasks including test planning, analysis, design, and implementation; reporting test progress and outcomes; and assessing a test object's quality.

5.2 Understanding Testing Methodologies: Exploring Dynamic and Static Testing Approaches

Dynamic testing is a type of testing in which the system or component being evaluated is really run. Static testing is another type of testing that doesn't include running the system or component under test. Thus, testing also entails evaluating outputs such user stories, specifications, and source code. Testing is sometimes misunderstood to be solely concerned with verifying requirements, user stories, or other specifications. Testing involves determining whether the system satisfies predetermined requirements, but it also entails validating whether the system will satisfy user and other requirements of stakeholders in the operational environment (s). In various lifecycles, test activities are structured and executed differently, here we will see the main levels of testing, such as black box, white box, and gray box testing.

5.2.1 Black Box Testing

Analysing the proper test basis such as formal requirements documents, specifications, use cases, user stories, or business processes is the foundation for black-box test approaches, also known as behavioural or behaviour-based techniques. These methods can be used for both nonfunctional and functional testing. Black box testing methods focus on the test object's inputs and outputs without considering its internal workings. Myers, G.J., Sandler, C., and Badgett, T. (2011).

The following are typical traits of black-box test methodologies:
• Software requirements, specifications, use cases, and user stories may comprise the test base from which test conditions, test cases, and test data are generated.
• Test cases can be used to identify discrepancies and inconsistencies between the requirements and how they are implemented.
• The items tested in the test basis and the methodology used to create the test basis are used to determine coverage.

Black-box Test Techniques:

- Equivalence partitioning: To derive test cases from given requirements.

Partitioning by equivalency separates data into partitions (sometimes called equivalency classes) with the expectation that each partition's members will be handled uniformly (Kaner 2013; Jorgensen 2014). Equivalency partitions are available for both valid and incorrect values.

Example: Imagine you're testing a login form. For the "username" field, valid values could be alphanumeric strings between 4 to 10 characters long, while invalid values could include special characters or strings longer than 10 characters. By testing both valid and invalid partitions separately, we ensure that the login form behaves correctly under various scenarios.

- Boundary value: Analysis to derive test cases from given requirements.

An extension of equivalency partitioning, boundary value analysis (BVA) is only applicable in ordered partitions with sequential or numeric data. A partition's border values are its minimum and maximum values, often known as its first and last values (Beizer 1990). When testing a system that calculates loan interest rates, you would focus on testing boundary values like the lowest and highest possible loan amounts to ensure accurate calculations and handling of edge cases.

This ensures the system behaves correctly when pushed to its limits.

Example: Consider the following scenario: a keypad is used to restrict inputs so that non-integer values are not available in an input field that only accepts a single integer number as an input.

- Decision table: testing to derive test cases from given requirements.

Making a table that links various input combinations to appropriate actions or outputs is the first step in decision table testing. This method is especially helpful in situations where a system's behaviour is governed by several conditions or rules. Test cases are created by methodically going over every possible combination of inputs and the decision table-defined consequences that go along with them.

Example: Testing a login system with different combinations of valid and invalid usernames and passwords to ensure proper authentication.

- State transition: testing to derive test cases from given requirements.

Testing a system's transitions between various states is the main goal of state transition testing. This method is frequently applied to finite-state systems, like user interfaces and state machines. To ensure that the system performs correctly as it transitions between states, test cases are created by first identifying the possible states of the system and the transitions between them.

Example: Testing a traffic light system to verify that it transitions correctly between "green," "yellow," and "red" states in response to sensor inputs and timing logic.

- How to derive test cases from a use case.

The process of building test cases to ensure that each scenario operates as intended begins with determining the numerous scenarios or paths that are represented in a use case. The actors engaged, how they interact with the system, and the anticipated results outlined in the use case can all be considered while creating test cases. To guarantee thorough coverage of the system's functionality as outlined in the use case, every step or action in the use case can be converted into a test case.
Example: Testing an online shopping checkout process based on a use case scenario, verifying steps like adding items to the cart, entering payment information, and completing the purchase. IEEE. (2014).

Black-Box Test Advantages:

- The application tester is not required to be a licenced software developer or to be familiar with the application code.

- The tester considers the operation of the application more.

Black-Box Test Disadvantages:

- There is a very slim probability of discovering the source of the mistake.
- There is no way of testing individual code processes.
- The directions to be checked are always restricted by the application's GUI.

5.2.2 White Box Testing

White-box testing methods, commonly referred to as structural or structure-based methods, are predicated on an examination of the test object's internal structure, architecture, detailed design, or code. White-box test methods focus on the organisation and processing of the test object, as opposed to black-box methods.
Experience-based test methodologies utilise the combined knowledge and skills of developers, testers, and users to create, organise, and carry out tests. These methods are frequently used in conjunction with white-box and black-box test methods. Myers, G.J., Sandler, C., and Badgett, T. (2011).

The following are typical traits of white-box test methodologies:

Statement coverage**:** Guarantees that, throughout testing, every statement in the code is run at least once. This helps to identify dead code or unexecuted branches by confirming that every section of the code can be accessed.
Example: Testing a function that calculates the area of a triangle to ensure that each line of code, such as variable assignments and mathematical operations, is executed during testing.

Decision coverage**:** Verifies that, at least once during testing, each decision point in the code such as if statements or loops evaluates to both true and false. This confirms that every scenario for a decision has been examined, which aids in identifying logical flaws or unmet requirements.

Example: Testing a login function with both valid and invalid credentials to ensure that all decision points, such as if statements determining login success or failure, are evaluated for both true and false conditions.

Combining statement and decision coverage**:** Assures that the requirements for both types of coverage are fulfilled. This offers a more thorough examination of the behaviour of the code by validating not just specific statements but also the logical progression and decision-making routes inside the code.
Example: Testing a sorting algorithm to verify that not only every line of code is executed (statement coverage), but also that every decision point, such as loops and conditional statements, evaluates to both true and false at least once (decision coverage). IEEE. (2014).

Advantages of White-Box Testing:

- The ability to check the different code portions.
- It may be simpler to identify the source of the mistake by checking.
- Testing process automation that happens quickly.
- Simplifies the process of customising the different code portions.

Disadvantages of White-Box Testing:

- It necessitates familiarity with the independently verified code.
- It can take a lot of time to prepare for the test.

5.2.3 Grey Box Testing

Grey-box testing methods, also known as translucent testing, involve a combination of both black-box and white-box testing techniques. This approach leverages partial knowledge of the system's internal workings, which can help design more effective test cases. Grey-box testing is used to identify defects due to improper code structure or improper usage of the application. It bridges the gap between black-box and white-box testing by combining both external user perspective and internal code structure.
Experience-based test methodologies utilize the combined knowledge and skills of developers, testers, and users to create, organize, and carry out tests. These methods are frequently used in conjunction with grey-box, black-box, and white-box test methods. Myers, G.J., Sandler, C., and Badgett, T. (2011).

The following are typical traits of grey-box test methodologies:

Combining functional and structural testing: Grey-box testing evaluates the system's functionality while considering certain aspects of its internal structure. This allows testers to create more comprehensive and effective test cases. Example: Testing a web application by checking both the user interface and some backend processes, such as database queries and server responses.

Partial code access***:*** In white-box testing. This helps in identifying issues that may not be visible through black box testing alone. Example: Testing an e-commerce application where the tester reviews the code related to payment processing but primarily focuses on end-to-end functionality from the user perspective.

Interface testing: Focuses on testing the interactions between different modules or components within the application, ensuring they work together as expected. Example: Testing an API where the tester understands the API endpoints and their expected inputs and outputs, verifying both the API's internal logic and its integration with the application. IEEE. (2014).

Advantages of Grey-Box Testing:

Balanced approach: Combines the benefits of both black-box and white-box testing, leading to more thorough testing coverage.

Enhanced test scenarios: Allows the creation of more effective test cases by using knowledge of the system's internal logic.

Improved defect identification: Can reveal issues related to both functionality and code structure, increasing the likelihood of finding defects. IEEE. (2014).

Disadvantages of Grey-Box Testing:

Limited internal knowledge: Testers do not have full access to the code, which can sometimes restrict the depth of structural testing.

Complexity: Requires testers to have skills in both black-box and white-box testing techniques, which can be challenging to find and train.

Time-consuming: Designing and executing grey-box tests can be more time-consuming compared to purely black-box or white-box testing due to the need to consider both perspectives. IEEE. (2014).

5.2.4 Chosen Method to Test the Current Project's Application

In the PDTracker project, Grey-Box Testing will be employed as a crucial validation and testing tool for the developed Android application. The importance of Grey-Box testing lies in its focus on both storyboarding and the application's specification features. By using the Grey-Box approach, a broader scope of the application can be covered compared to using White-Box or Black-Box methods alone. While White-Box testing emphasizes the software code and Black-Box testing targets the user's functional requirements, Grey-Box testing combines both perspectives. This ensures that all necessary specifications are thoroughly planned and enforced. Moreover, due to the time constraints of the PDTracker project, Grey-Box testing is particularly suitable. It integrates both White-Box and Black-Box testing techniques, facilitating the identification and resolution of bugs and inaccuracies before the final system implementation.

The main techniques we use to test our app:

Error Guessing: We used the knowledge of past application behaviour, common error types, and failures in similar applications to anticipate errors. We specifically design test cases to discover these potential flaws and defects. Exploratory Testing: Informal tests are dynamically designed, executed, and evaluated during testing. The test results are used to guide further testing areas. Exploratory testing was structured using session-based testing, where testing is conducted within defined time-boxes and guided by test charters and session sheets for documentation. Checklist-based Testing: We implemented and execute tests based on predefined checklists. These checklists, which can be modified or used as is, are created from experience or knowledge about user needs and common failure scenarios. They support various test types, providing guidelines and consistency in testing, although with some variability and potentially greater coverage but less repeatability. By leveraging Grey-Box testing in the PDTracker project, we aim to achieve a robust, reliable, and user-friendly Android application.

We also have a plan for automated tests in the future. It is important to highlight that regression tests can be automated and executed before each deployment in production when the system is updated with new features. One of the biggest challenges is including test automation within a short sprint for each task. However, we can manage this by using a tool to automate only the new features that really need it. This means focusing on the main flow so that we have an effective tool working end-to-end with each deployment. IEEE. (2014).

5.3 Application Testing Process with Testing Cases – PDTracker Application Testing Process.

1. ***Planning Phase***

Objective: Define the scope, objectives, and approach for testing.

Scope: Determine the features and functionalities of PDTracker to be tested.

Objectives: Ensure PDTracker meets the requirements and functions correctly.

Approach: Choose testing types (functional, non-functional, regression, etc.).

2. ***Design Phase***

Objective: Develop test cases and scenarios.

Test Case Development: Create detailed test cases covering all features.

Test Environment Setup: Prepare the environment for testing.

Test Data Preparation: Generate necessary data for test execution.

3. ***Execution Phase***

Objective: Execute test cases and report issues.

Test Execution: Run the test cases on the prepared environment.

Defect Reporting: Log defects in a tracking system.

Retesting and Regression Testing: Ensure fixes do not break existing functionality.

4. ***Closure Phase***

Objective: Finalize testing and report on outcomes.

Test Summary Report: Document the testing process and results.

Defect Analysis: Review and analyze defects found during testing.

Sign-off: Obtain approval from stakeholders to release the application.

5.3.1 Errors Encountered During the Application Coding Process

During the Coding process of the application, few initial errors were detected and corrected.

Table 5.1 displays some of the errors that have been found and dully corrected.

Table 5.1: Initial Errors detected during the Coding Process.



test suite	Precondition:	Test data	Test cases	Step by step collected with evidence	Expected Result	Actual Result	Pass	Fail
Patient_LoginScreen(Main flow)	"Have a registered user in the system with a valid email."	"Email successfully registered in the database."	"Recover the password forgotten"	1. Go in to the application and click on "Patient." 2. Click on the "Forgotten Password" link. 3. Check the email in the database to reset the password.	"The password reset link is sent to the registered email correctly.	The system does not send the password reset email to the registered email.	✓	

test suite	Precondition:	Test data	Test cases	Step by step collected with evidence	Expected Result	Actual Result	Pass	Fail
Patient_LoginScreen(Main flow)	"Have a registered user in the system with a valid email."	"Email successfully registered in the database."	"Recover the password forgotten"	1. Go in to the application and click on "Patient." 2. Click on the "Forgotten Password" link. 3. Check the email in the database to reset the password.	"The password reset link is sent to the registered email correctly.	The system does not send the password reset email to the registered email.	✓	

Patients_RegScreen (Negative test)	"Have a invalid Irish phone number."	Invalid phone number	"Register user with an invalid Irish phone number."	1. Access the app and click on "Register Here." 2. "Fill in all the mandatory fields on the screen with an invalid irish number. 3. Click in register	"The system does not register the user with an invalid phone number and displays an error message upon completion of typing."	"The system registers the user with an invalid phone number."	✓	
Patients_RegScreen (Negative test)	"Have patient with the age less than 18"	"Patient with the age less than 18"	"Register a user under 18 years of age."	1. Access the app and click on "Register Here." 2. "Fill in all the mandatory fields on the screen with a year in DOB field more than 2007 3. Click in register	"The system should show the message> YOU MUST BE AT LEAST 18 YEARS OLD TO REGISTER.	"The system registers the user and don't show the error message	✓	



test suite	Precondition:	Test data	Test cases	Step by step collected with evidence	Expected Result	Actual Result	Pass	Fail
Patients_RegScreen (Negative test)	"Have the 'Password' field registered correctly."	N / A	"Register user with incorrect 'Re-type password' field."	1. Access the app and click on "Register Here." 2. "Fill in all the mandatory fields on the screen with the field re-type password wrong. 3. Click in register	"The system should show the message> Password do not match.	"The system registers the user and don't show the error message	✓	
Patients_Patient Profile (Main flow- Positive test)	"Have a smartwatch ID registered in the application." "Have a successfully registered user."	ID SMARTWATCH valid	Consult the synchronization and validate the information coming from the smartwatch for the 'Daily Data Reading' screen."	1. Access the system, fill in the login and password fields, click on login. 2. Click on 'Create/Update Patient Profile,' fill in all the information on the screen, and click save. 3.Click on 'Daily Data from Smartwatch' and click sync.	"The system show the informations: Normal Blood Pressure:" "Threshould Blood Pressure: (Y/N)" "Heart Rate Abnormality: Yes" "Oxygen Levels: " "Sleep Quality:" "Fall Detected: ?" "GPS" "Heart Rate Details"	The system displays the message: 'No health data found for the smartwatch ID.'"	✓	

Test Case	Expected Result	Actual Result	Fixed?
"Recover the password forgotten"	"The password reset link is sent to the registered email correctly.	The system does not send the password reset email to the registered email.	✓
"Register user with an invalid Irish phone number."	"The system does not register the user with an invalid phone number and displays an error message upon completion of typing."	"The system registers the user with an invalid phone number."	✓

"Register user with incorrect 'Re-type password' field."	"The system should show the message> Password do not match.	"The system registers the user and don't show the error message.	✓
Consult the synchronization and validate the information coming from the smartwatch for the 'Daily Data Reading' screen."	"The system show the informations: Normal Blood Pressure:" "Threshould Blood Pressure: (Y/N)" "Heart Rate Abnormality: Yes" "OxygenLevels: " "SleepQuality:" "Fall:?" "GPS" "Heart Rate Details"	The system displays the message: 'No health data found for the smartwatch ID.'"	✓

5.3.2 Functional Testing

This employs Black box testing methodology, focusing on evaluating software functional requirements. The assessment involves providing inputs, analysing feedback, and verifying if the outcomes align with the application's intended features. Functional testing occurs on a fully assembled system to ascertain its adherence to specified requirements. The program's functionality is scrutinized throughout its development stages until deployment. The objective is to verify that the application functions according to the criteria outlined in the current project's Use Case Diagram. Functional testing can encompass main flow and alternative flow, as well as negative testing, positive testing, regression testing, and domain testing. IEEE. (2014).

Table 5.2: Functional Testing Process with Testing Cases

Function: (Home Dashboard) & (Patient Registration, Login, Dashboard, Profile and Sync data, Notification from Doctor).				
Test Case	**Expected Result**	**Pass**	**Fail**	**Actual Result**
Access 'Patient' functionality from the home dashboard	Redirected to the 'Patient' screen with the correct functionality	Pass		Redirected to the 'Patient' screen with the correct functionality

Access 'Doctor' functionality from the home dashboard	Redirected to the 'Doctor' screen with the correct functionality	Pass		Redirected to the 'Doctor' screen with the correct functionality
Access 'Caregiver' functionality from the home dashboard	Redirected to the 'Caregiver' screen with the correct functionality	Pass		Redirected to the 'Caregiver' screen with the correct functionality
Register patient with a phone number that is not in the Irish format (10 digits in the field domain)	Error message indicating invalid phone number format	Pass		Error message indicating invalid phone number format
Register patient with an age under 18 years	Error message indicating invalid age	Pass		Error message indicating invalid age
Register patient with an invalid email format	Error message indicating invalid email format	Pass		Error message indicating invalid email format
Register patient with a weak password	Error message indicating weak password	Pass		Error message indicating weak password
Register patient with incorrect re-type password	Error message indicating password mismatch	Pass		Error message indicating password mismatch
Register patient with all valid data	Success message indicating successful registration	Pass		Success message indicating successful registration
Log in with an invalid email	Error message indicating invalid email	Pass		Error message indicating invalid email
Log in with an invalid password	Error message indicating invalid password	Pass		Error message indicating invalid password
Log in with valid username and password	Success message indicating successful login	Pass		Success message indicating successful login
Reset password for registered user	Success message indicating password reset email sent	Pass		Success message indicating password reset email sent
Reset password for non-registered user	Error message indicating user not found	Pass		Error message indicating user not found
Create/update patient profile with a patient weighing less than 40kg	Error message indicating invalid weight	Pass		Error message indicating invalid weight
Create/update patient profile with a patient weighing more than 150kg	Error message indicating invalid weight	Pass		Error message indicating invalid weight
Create/update patient profile with a heart rate less than 60 or more than 100	Error message indicating invalid heart rate range	Pass		Error message indicating invalid heart rate range
Create/update patient profile with a blood pressure threshold less than 180	Error message indicating invalid blood pressure threshold	Pass		Error message indicating invalid blood pressure threshold
Create/update patient profile with	Error message indicating invalid blood pressure range	Pass		Error message indicating invalid blood pressure range

blood pressure in the range of 120/40				
Create/update patient profile with a blood pressure threshold out of the range of 200/40	Error message indicating invalid blood pressure threshold	Pass		Error message indicating invalid blood pressure threshold
Create/update patient profile with an invalid smartwatch ID (not registered in the database or invalid ID)	Error message indicating invalid smartwatch ID	Pass		Error message indicating invalid smartwatch ID
Consult health data on the 'Daily Data from Smartwatch' screen by clicking sync	Error message indicating no health data found for the smartwatch ID	Pass		Error message indicating no health data found for the smartwatch ID
Create/update patient profile with all correct data	Success message indicating profile updated successfully	Pass		Success message indicating profile updated successfully
Consult health data on the 'Daily Data from Smartwatch' screen by clicking sync	Health data successfully synced and displayed	Pass		Health data successfully synced and displayed
View notifications after an abnormality	Notifications displayed successfully	Pass		Notifications displayed successfully
Function: (Doctor Registration, Login, Dashboard, Profile and Sync data, Notification from Patient).				
Doctor's Home (Dashboard) Screen	Doctors can access essential functionalities for patient management from the dashboard.	Pass		Doctors access essential functionalities for patient management from the dashboard.
Create/Update Emergency Notes For Patient Screen	Doctors can manage and respond to emergency situations by creating and updating notifications.	Pass		Doctors manage and respond to emergency situations by creating and updating notifications.
View Daily Data Readings from Patients' Smartwatch Screen	Doctors can monitor patients' daily health metrics and data collected from smartwatches effectively.	Pass		Doctors monitor patients' daily health metrics and data collected from smartwatches effectively.

View Alert of Patient Abnormal Condition Detected & Send Notification to Patient and Caregiver for Action Screen	Doctors can view alerts for abnormal conditions detected in patients and send notifications to both the Patient and their Caregiver with the necessary actions to be taken.	Pass		Doctors view alerts for abnormal conditions detected in patients and send notifications to both the Patient and their Caregiver with the necessary actions to be taken.
View Selected Alert Details and Send Notification to Patient and Caregiver Screen	Doctors can view detailed information about a selected alert concerning a patient's abnormal condition and send notifications with specific instructions to both the Patient and their Caregiver.	Pass		Doctors view detailed information about a selected alert concerning a patient's abnormal condition and send notifications with specific instructions to both the Patient and their Caregiver.
Create Notification to Patient and Caregiver with Actions to Follow Screen	Doctors can create and send notifications to Patients and their Caregivers, detailing specific actions to follow in response to a detected abnormal condition.	Pass		Doctors create and send notifications to Patients and their Caregivers, detailing specific actions to follow in response to a detected abnormal condition.
View Alert of Patient Fall Detected and Contact Patient, Caregiver and Paramedics Screen	Doctors can view alerts for detected patient falls and contact the Patient, Caregiver, and Paramedics immediately.	Pass		Doctor's view alerts for detected patient falls and contact the Patient, Caregiver, and Paramedics immediately.
Send Doctors Emergency Notification to Paramedics Screen	Doctors can send emergency notifications directly to Paramedics in response to critical situations involving patients.	Pass		Doctors send emergency notifications directly to Paramedics in response to critical situations involving patients.
Function: (Caregiver Registration, Login, Dashboard, Profile and Sync data, Notification from Patient).				
Caregiver's Login Screen	Caregivers can log in using their credentials and access the application's functionalities tailored to their role.	Pass		Caregivers log in using their credentials and access the application's,functionalities tailored to their role.

Create/Update Caregiver Account Screen	Caregivers can successfully register and create an account, providing necessary personal and professional information.	Pass		Caregivers successfully register and create an account, providing necessary personal and professional information.
Caregiver's Home (Dashboard) Screen	Caregivers can access essential functionalities for patient management from the dashboard.	Pass		Caregivers access essential functionalities for patient management from the dashboard.
Create/Update Emergency Notes For Patient Screen	Caregivers can manage and respond to emergency situations by creating and updating notifications.	Pass		Caregivers manage and respond to emergency situations by creating and updating notifications.
View Daily Data Readings from Patients' Smartwatch Screen	Caregivers can monitor patients' daily health metrics and data collected from smartwatches effectively.	Pass		Caregivers monitor patients' daily health metrics and data collected from smartwatches effectively.
View Doctor's Notifications for Patient Abnormal Conditions with Instructions to Act Screen	Caregivers can receive notifications from doctors about abnormal conditions and follow provided instructions.	Pass		Caregivers receive notifications from doctors about abnormal conditions and follow provided instructions.
View Patient Fall Detection Alerts Screen	Caregivers can receive real-time alerts for patient falls and take immediate action.	Pass		Caregivers receive real-time alerts for patient falls and take immediate action.

5.3.3 Non-Functional Testing

The foundation of this kind of testing is determining whether non-functional software requirements are met. Testing apps for non-functional but essential needs, such as their performance, security, usability, and navigation, is known as non-functional testing.

Performance Testing

Performance testing is a crucial and required technique for evaluating the following aspects: software scalability, capability, stability, and growth, and speed (i.e., response time, data rendering and access). Rather than discovering bugs in programmes, it is often used to identify bottlenecks or operating issues.

Table 5.3: Performance Testing Process with Testing Cases

Test Case	Expected Result	Pass	Fail	Actual Result
Evaluate Login and Logout Speed	Login and Logout processes are swift and responsive.	✓		Processes execute quickly without delays.
Verify Screen Loading	All screens load seamlessly without errors.	✓		Screens load promptly and error-free.
Test Record Update Efficiency	Changes in storage records reflect immediately on the Client's side.	✓		Record updates are mirrored promptly on the Client's side.
Assess Application Stability	The application operates smoothly without crashes or abnormal behavior, utilizing memory efficiently.	✓		The application remains stable and responsive, not consuming excessive device memory.
Validate User Interface Alerts	All notifications are delivered accurately to users.	✓		Users receive timely alerts through the notification bar.

Usability Testing

Usability testing is a Black-box technique that watches users while they interact with an application to find any errors or changes in the device. The programme designed for ease of use will achieve five key goals: readability, ease of recall, productivity, suitability, and ease of comprehension.

Table 5.4: Usability Testing Process with Testing Cases for the User Interface

Test Case	Expected Result	Pass	Fail	Actual Result
Check Home Screen Functionality	The Home Screen provides clear user options and transitions seamlessly to the Login screen.	✓		Users navigate effortlessly from the Home Screen to the Login screen.
Validate User Login Process	The Login screen functions smoothly, granting access upon valid credentials.	✓		Users can log in successfully with correct credentials.
Test Registration Link	The Registration link directs users to the Registration screen for new account creation.	✓		Users can access the Registration screen from the Login page.
Verify Forgotten Password Link	The Forgotten Password link redirects users to the Reset Password screen for password recovery.	✓		Users can initiate password reset from the Login screen.
Assess Dashboard Functionality	The User Type Dashboard displays relevant information and functions as intended post-login.	✓		Users land on their respective Dashboards with appropriate access.

Security Testing

Security monitoring involves encrypting sensitive data, including user passwords, erasing user sessions via a logout procedure, and testing apps to find any bugs or security issues in terms of confidentiality, availability, and honesty.

Table 5.5: Security Testing Process with Testing Cases

Test Case	Expected Result	Pass	Fail	Actual Result
Validate Password Encryption	User passwords are stored securely in encrypted form.	✓		Passwords are securely encrypted in the application database.
Verify User Session Clearance	User sessions are terminated, and data cleared upon logout.	✓		User sessions end correctly, clearing user data upon logout.
Test User Data Protection	User information remains confidential and inaccessible to unauthorized parties.	✓		User data is protected and not accessible to third parties without consent.

5.4 Chapter summary

This chapter delved into the prevalent software testing practices in the industry, specifically focusing on Black-box, White-box, and Gray-box testing methods. By examining their respective advantages and limitations, Gray-box Testing emerged as the optimal choice for the application under development. This method leverages the strengths of both Black-box and White-box testing, ensuring a comprehensive evaluation of the software. During the development of the application, rigorous testing was conducted. The initial coding phase revealed a few minor errors, which were promptly and thoroughly corrected. To ensure the application met all specified requirements and performed flawlessly, both functional and non-functional tests were executed.

5.5 References

Farley, R. and Humphrey, W.S., 2006. The Capability Maturity Model: Guidelines for Improving the Software Process. Boston: Addison-Wesley.

IEEE, 2014. IEEE Standard for Software and System Test Documentation. IEEE Std 829-2008. Available at: https://standards.ieee.org/standard/829-2008.html [Accessed 22 May 2024].

McGraw, G. and Felten, E.W., 2006. Securing Java: Getting Down to Business with Mobile Code. New York: Wiley.

Miller, E., 2012. Software Testing Techniques. 2nd ed. Cambridge: MIT Press.

Myers, G.J., Sandler, C. and Badgett, T., 2011. The Art of Software Testing. 3rd ed. New York: John Wiley & Sons.

Nielsen, J., 1993. Usability Engineering. San Diego: Academic Press.

Pressman, R.S., 2014. Software Engineering: A Practitioner's Approach. 8th ed. New York: McGraw-Hill.

Sommerville, I., 2016. Software Engineering. 10th ed. Boston: Pearson.